Compulsory Assignment 5: Sentiment Analysis

Please fill out the the group name, number, members and optionally the name below.

Group number:

Member 1: Oliver Tschudi Madsen

Member 2: Mehdi Jamali

Member 3: Tobias Tessem Hole

Group name (optional): "Swagsters"

Assignment Submission

To complete this assignment answer the questions in this notebook and write the code required to implement the relevant models. However, we reccomend that you start early. This assignment has subsections, each of which build upon the last. This assignment is completed by doing the following.

- Submit notebook as an .ipynb file to canvas.
- Submit notebook as an .pdf file to canvas.

Your submission must correctly implement the full pipeline to analyse the sentiment using a RNN and BERT models.

NOTE: It is recommended to use Google Colab with GPU enabled for CA5 tasks. Keras and TensorFlow (version ≥2) will automatically use the GPU if one is available

- Read tasks thoroughly before answering.
- Make sure to answer all tasks.
- Ensure all code cells are run.
- Ensure all figures are visible in the PDF.
- Provide a brief explanation of how your code works

How to Use Google Colab for CA5

To work on the CA5 assignment in Google Colab, follow these steps:

---

1. Connect Google Colab to Google Drive

Your CA5 files should be stored in Google Drive so Colab can access them.
Run the following code to mount your Drive:

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
```

```
import os
project_path = "/content/drive/MyDrive" #change this to your folder.
os.chdir(project_path)
print("Current working directory:", os.getcwd())
```

```
Current working directory: /content/drive/MyDrive
```

Change Runtime to GPU

To use GPU in Google Colab:

1. Go to the top menu and click **Runtime** → **Change runtime type**.
2. In the pop-up window, set **Hardware accelerator** to **GPU**.
3. Click **Save**.

Now your notebook will run on a GPU backend.

###**Note**: I recommend using the regular runtime first, as your GPU usage is limited. Write and test your code in the regular runtime to ensure everything works correctly before switching to the GPU for model training.

##Introduction In this assignment, you will analyze whether a movie review is positive or negative. In Task 1, you will use a Recurrent Neural Network (RNN) to perform the analysis. In Task 2, you will be introduced to the encoder block of the Transformer architecture, commonly known as BERT. Finally, you will compare the results from both approaches and draw a conclusion.

Note: BERT stands for "Bidirectional Encoder Representations from Transformers"

###About the Dataset The IMDb dataset is a collection of 50,000 reviews from the Internet Movie Database (IMDb). The reviews are labeled as either positive or negative.

The IMDb dataset is a binary sentiment analysis dataset for natural language processing or text analytics.

IMDb is a rich source of film data that includes cast and crew lists, movie release dates, box office information, plot summaries, trailers, actor and director biographies, and other trivia. Information on IMDb comes from a variety of sources, such as filmmakers, film studios, on-screen credits, and other official sources.

Task 0: Load and Inspect the Data

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("IMDB Dataset.csv")

data.head()
```


```
{"summary":"{\n  \"name\": \"data\",\n  \"rows\": 50000,\n
\"fields\": [\n    {\n      \"column\": \"review\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 49582,\n        \"samples\": [\n
\"\\\"Soul Plane\\\" is a horrible attempt at comedy that only should
appeal people with thick skulls, bloodshot eyes and furry pawns.
<br /><br />The plot is not only incoherent but also non-existent,
acting is mostly sub sub-par with a gang of highly moronic and
dreadful characters thrown in for bad measure, jokes are often spotted
miles ahead and almost never even a bit amusing. This movie lacks any
structure and is full of racial stereotypes that must have seemed old
even in the fifties, the only thing it really has going for it is some
pretty ladies, but really, if you want that you can rent something
from the \\\"Adult\\\" section. OK?<br /><br />I can hardly see
anything here to recommend since you'll probably have a lot a better
and productive time chasing rats with a sledgehammer or inventing
waterproof teabags or whatever.<br /><br />2/10\",\n          \"Guest
from the Future tells a fascinating story of time travel, friendship,
battle of good and evil -- all with a small budget, child actors, and
few special effects. Something for Spielberg and Lucas to learn
from. ;) A sixth-grader Kolya \\\"Nick\\\" Gerasimov finds a time
machine in the basement of a decrepit building and travels 100 years
into the future. He discovers a near-perfect, utopian society where
robots play guitars and write poetry, everyone is kind to each other
and people enjoy everything technology has to offer. Alice is the
daughter of a prominent scientist who invented a device called
Mielophone that allows to read minds of humans and animals. The device
can be put to both good and bad use, depending on whose hands it falls
into. When two evil space pirates from Saturn who want to rule the
universe attempt to steal Mielophone, it falls into the hands of 20th
century school boy Nick. With the pirates hot on his tracks, he
travels back to his time, followed by the pirates, and Alice. Chaos,
confusion and funny situations follow as the luckless pirates try to
blend in with the earthlings. Alice enrolls in the same school Nick
goes to and demonstrates superhuman abilities in PE class. The catch
is, Alice doesn't know what Nick looks like, while the pirates do.
Also, the pirates are able to change their appearance and turn
literally into anyone. (Hmm, I wonder if this is where James Cameron
got the idea for Terminator...) Who gets to Nick -- and Mielophone --
first? Excellent plot, non-stop adventures, and great soundtrack. I
wish Hollywood made kid movies like this one...\",\n
```

\"\\\"National Treasure\\\" (2004) is a thoroughly misguided hodge-podge of plot entanglements that borrow from nearly every cloak and dagger government conspiracy clich\\u00e9 that has ever been written. The film stars Nicholas Cage as Benjamin Franklin Gates (how precious is that, I ask you?); a seemingly normal fellow who, for no other reason than being of a lineage of like-minded misguided fortune hunters, decides to steal a 'national treasure' that has been hidden by the United States founding fathers. After a bit of subtext and background that plays laughably (unintentionally) like Indiana Jones meets The Patriot, the film degenerates into one misguided whimsy after another \\u0096 attempting to create a 'Stanley Goodspeed' regurgitation of Nicholas Cage and launch the whole convoluted mess forward with a series of high octane, but disconnected misadventures.

The relevancy and logic to having George Washington and his motley crew of patriots burying a king's ransom someplace on native soil, and then, going through the meticulous plan of leaving clues scattered throughout U.S. currency art work, is something that director Jon Turteltaub never quite gets around to explaining. Couldn't Washington found better usage for such wealth during the start up of the country? Hence, we are left with a mystery built on top of an enigma that is already on shaky ground by the time Ben appoints himself the new custodian of this untold wealth. Ben's intentions are noble \\u0096 if confusing. He's set on protecting the treasure. For who and when?\\u0085your guess is as good as mine.

But there are a few problems with Ben's crusade. First up, his friend, Ian Holmes (Sean Bean) decides that he can't wait for Ben to make up his mind about stealing the Declaration of Independence from the National Archives (oh, yeah \\u0096 brilliant idea!). Presumably, the back of that famous document holds the secret answer to the ultimate fortune. So Ian tries to kill Ben. The assassination attempt is, of course, unsuccessful, if overly melodramatic. It also affords Ben the opportunity to pick up, and pick on, the very sultry curator of the archives, Abigail Chase (Diane Kruger). She thinks Ben is clearly a nut \\u0096 at least at the beginning. But true to action/romance form, Abby's resolve melts quicker than you can say, \\\"is that the Hope Diamond?\\\" The film moves into full X-File-ish mode, as the FBI, mistakenly believing that Ben is behind the theft, retaliate in various benign ways that lead to a multi-layering of action sequences reminiscent of Mission Impossible meets The Fugitive. Honestly, don't those guys ever get 'intelligence' information that is correct? In the final analysis, \\\"National Treasure\\\" isn't great film making, so much as it's a patchwork rehash of tired old bits from other movies, woven together from scraps, the likes of which would make IL' Betsy Ross blush.

The Buena Vista DVD delivers a far more generous treatment than this film is deserving of. The anamorphic widescreen picture exhibits a very smooth and finely detailed image with very rich colors, natural flesh tones, solid blacks and clean whites. The stylized image is also free of blemishes and digital enhancements. The audio is 5.1 and delivers a nice sonic

```
boom to your side and rear speakers with intensity and realism. Extras include a host of promotional junket material that is rather deep and over the top in its explanation of how and why this film was made. If only, as an audience, we had had more clarification as to why Ben and co. were chasing after an illusive treasure, this might have been one good flick. Extras conclude with the theatrical trailer, audio commentary and deleted scenes. Not for the faint-hearted \\u0096 just the thick-headed.\"\n        ],\n        \"semantic_type\": \"\",\n \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"sentiment\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"negative\",\n          \"positive\"\n        ],\n \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe","variable_name":"data"}

#code cell below was messing up and chatgpt found this solution, run if runtimeerror on next cell

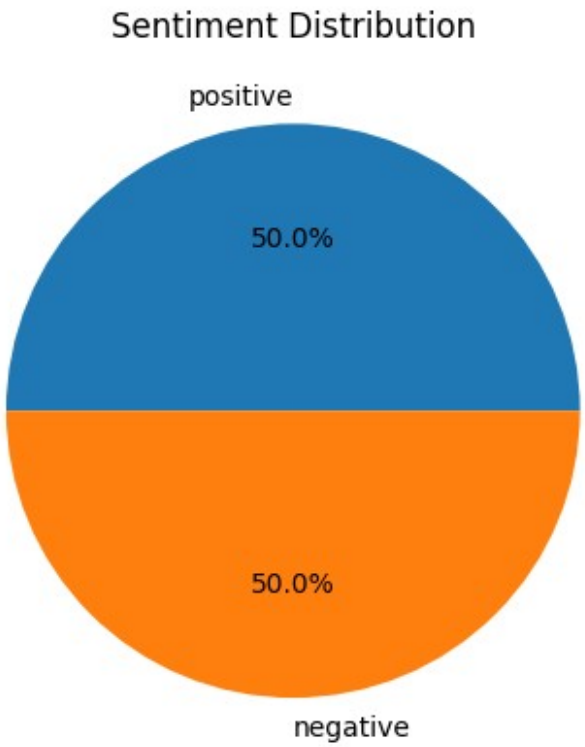
# import matplotlib as mpl
# import os

# cache_dir = mpl.get_cachedir()
# print("Cache directory:", cache_dir)

# # remove the cache
# import shutil
# shutil.rmtree(cache_dir, ignore_errors=True)

# print("Cache cleared. Restart kernel now.")

sentiment_counts = data["sentiment"].value_counts()
plt.pie(sentiment_counts, labels=sentiment_counts.index,
autopct='%1.1f%%')
plt.title("Sentiment Distribution")
plt.show()
```



Preprocess

```
from sklearn.preprocessing import LabelEncoder
import re
from sklearn.model_selection import train_test_split

labels = data["sentiment"]
enc = LabelEncoder()
labels_encoded = enc.fit_transform(labels)
data["labels_encoded"] = labels_encoded
data.head()
```


```
{"summary":"{\n  \"name\": \"data\",\n  \"rows\": 50000,\n
\"fields\": [\n    {\n      \"column\": \"review\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 49582,\n        \"samples\": [\n
\"\\\"Soul Plane\\\" is a horrible attempt at comedy that only should
appeal people with thick skulls, bloodshot eyes and furry pawns.
<br /><br />The plot is not only incoherent but also non-existent,
acting is mostly sub sub-par with a gang of highly moronic and
dreadful characters thrown in for bad measure, jokes are often spotted
miles ahead and almost never even a bit amusing. This movie lacks any
structure and is full of racial stereotypes that must have seemed old
even in the fifties, the only thing it really has going for it is some
pretty ladies, but really, if you want that you can rent something
from the \\\"Adult\\\" section. OK?<br /><br />I can hardly see
```

anything here to recommend since you'll probably have a lot a better and productive time chasing rats with a sledgehammer or inventing waterproof teabags or whatever.

2/10\",\n \"Guest from the Future tells a fascinating story of time travel, friendship, battle of good and evil -- all with a small budget, child actors, and few special effects. Something for Spielberg and Lucas to learn from. ;) A sixth-grader Kolya \\\"Nick\\\" Gerasimov finds a time machine in the basement of a decrepit building and travels 100 years into the future. He discovers a near-perfect, utopian society where robots play guitars and write poetry, everyone is kind to each other and people enjoy everything technology has to offer. Alice is the daughter of a prominent scientist who invented a device called Mielophone that allows to read minds of humans and animals. The device can be put to both good and bad use, depending on whose hands it falls into. When two evil space pirates from Saturn who want to rule the universe attempt to steal Mielophone, it falls into the hands of 20th century school boy Nick. With the pirates hot on his tracks, he travels back to his time, followed by the pirates, and Alice. Chaos, confusion and funny situations follow as the luckless pirates try to blend in with the earthlings. Alice enrolls in the same school Nick goes to and demonstrates superhuman abilities in PE class. The catch is, Alice doesn't know what Nick looks like, while the pirates do. Also, the pirates are able to change their appearance and turn literally into anyone. (Hmm, I wonder if this is where James Cameron got the idea for Terminator...) Who gets to Nick -- and Mielophone -- first? Excellent plot, non-stop adventures, and great soundtrack. I wish Hollywood made kid movies like this one...\",\n \"\\\"National Treasure\\\" (2004) is a thoroughly misguided hodge-podge of plot entanglements that borrow from nearly every cloak and dagger government conspiracy clich\\u00e9 that has ever been written. The film stars Nicholas Cage as Benjamin Franklin Gates (how precious is that, I ask you?); a seemingly normal fellow who, for no other reason than being of a lineage of like-minded misguided fortune hunters, decides to steal a 'national treasure' that has been hidden by the United States founding fathers. After a bit of subtext and background that plays laughably (unintentionally) like Indiana Jones meets The Patriot, the film degenerates into one misguided whimsy after another \\u0096 attempting to create a 'Stanley Goodspeed' regurgitation of Nicholas Cage and launch the whole convoluted mess forward with a series of high octane, but disconnected misadventures.

The relevancy and logic to having George Washington and his motley crew of patriots burying a king's ransom someplace on native soil, and then, going through the meticulous plan of leaving clues scattered throughout U.S. currency art work, is something that director Jon Turteltaub never quite gets around to explaining. Couldn't Washington found better usage for such wealth during the start up of the country? Hence, we are left with a mystery built on top of an enigma that is already on shaky ground by the time Ben appoints himself the new custodian of this untold wealth. Ben's

intentions are noble \\u0096 if confusing. He's set on protecting the treasure. For who and when?\\u0085your guess is as good as mine.

But there are a few problems with Ben's crusade. First up, his friend, Ian Holmes (Sean Bean) decides that he can't wait for Ben to make up his mind about stealing the Declaration of Independence from the National Archives (oh, yeah \\u0096 brilliant idea!). Presumably, the back of that famous document holds the secret answer to the ultimate fortune. So Ian tries to kill Ben. The assassination attempt is, of course, unsuccessful, if overly melodramatic. It also affords Ben the opportunity to pick up, and pick on, the very sultry curator of the archives, Abigail Chase (Diane Kruger). She thinks Ben is clearly a nut \\u0096 at least at the beginning. But true to action/romance form, Abby's resolve melts quicker than you can say, \\\"is that the Hope Diamond?\\\" The film moves into full X-File-ish mode, as the FBI, mistakenly believing that Ben is behind the theft, retaliate in various benign ways that lead to a multi-layering of action sequences reminiscent of Mission Impossible meets The Fugitive. Honestly, don't those guys ever get 'intelligence' information that is correct? In the final analysis, \\\"National Treasure\\\" isn't great film making, so much as it's a patchwork rehash of tired old bits from other movies, woven together from scraps, the likes of which would make IL' Betsy Ross blush.

The Buena Vista DVD delivers a far more generous treatment than this film is deserving of. The anamorphic widescreen picture exhibits a very smooth and finely detailed image with very rich colors, natural flesh tones, solid blacks and clean whites. The stylized image is also free of blemishes and digital enhancements. The audio is 5.1 and delivers a nice sonic boom to your side and rear speakers with intensity and realism. Extras include a host of promotional junket material that is rather deep and over the top in its explanation of how and why this film was made. If only, as an audience, we had had more clarification as to why Ben and co. were chasing after an illusive treasure, this might have been one good flick. Extras conclude with the theatrical trailer, audio commentary and deleted scenes. Not for the faint-hearted \\u0096 just the thick-headed.\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"sentiment\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 2,\n   \"samples\": [\n   \"negative\",\n   \"positive\"\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"labels_encoded\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0,\n   \"min\": 0,\n   \"max\": 1,\n   \"num_unique_values\": 2,\n   \"samples\": [\n   0,\n   1\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"data"}

```
def clean_text(text):
    text = re.sub(r"<br\s*/?>", " ", text)  # remove <br> or <br />
```

```
    text = re.sub(r" ", " ", text)
    text = re.sub(r"http\S+", " ", text)    # remove URLs
    text = re.sub(r"@\w+", " ", text)       # remove @mentions
    text = re.sub(r"[^A-Za-z0-9\s.,!?']", " ", text)# strip symbols
    text = re.sub(r"\s+", " ", text).strip()
    return text

data["review"] = data["review"].apply(clean_text)

# ensure labels_encoded exists (avoid KeyError if the preprocessing
cell wasn't run)
if "labels_encoded" not in data.columns:
     enc = LabelEncoder()
     data["labels_encoded"] = enc.fit_transform(data["sentiment"])

X_train, X_test, y_train, y_test = train_test_split(
     data["review"], data["labels_encoded"], test_size=0.25,
random_state=42
)
```

Task 1: RNN

1.1 Vectorize inputs

```
import tensorflow as tf
from tensorflow.keras.layers import TextVectorization
from tensorflow.keras import layers, Sequential

max_tokens = 30000
max_len = 128

vectorizer = TextVectorization(
    max_tokens=max_tokens,
    output_mode='int',
    output_sequence_length=max_len
)

vectorizer.adapt(tf.data.Dataset.from_tensor_slices(X_train).batch(102
4))

X = vectorizer(X_train)
```

Task 1.1 Create a simple RNN and an LSTM model

```
simple_rnn_model = Sequential([
    layers.Embedding(input_dim=max_tokens, output_dim=64,
mask_zero=True),
    layers.SimpleRNN(64),
    layers.Dense(1, activation='sigmoid')
])
```

```
simple_rnn_model.compile(
    loss = 'binary_crossentropy',
    optimizer = 'adam',
    metrics = ['accuracy']
)

simple_rnn_model.summary()

lstm_model = Sequential([
    layers.Embedding(input_dim=max_tokens, output_dim=64,
mask_zero=True),
    layers.LSTM(64),
    layers.Dense(1, activation='sigmoid')
])

lstm_model.compile()
loss = 'binary_crossentropy',
optimizer = 'adam',
metrics = ['accuracy']

lstm_model.summary()
```

```
Model: "sequential"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
┩
│ embedding (Embedding)                │ ?                           │   0
(unbuilt) │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ simple_rnn (SimpleRNN)               │ ?                           │   0
(unbuilt) │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ dense (Dense)                        │ ?                           │   0
(unbuilt) │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────
┘
```

```
 Total params: 0 (0.00 B)

 Trainable params: 0 (0.00 B)

 Non-trainable params: 0 (0.00 B)
```

```
Model: "sequential_1"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━
┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━
┩
│ embedding_1 (Embedding)              │ ?                           │   0
(unbuilt) │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ lstm (LSTM)                          │ ?                           │   0
(unbuilt) │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────
┤
│ dense_1 (Dense)                      │ ?                           │   0
(unbuilt) │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────
┘

 Total params: 0 (0.00 B)

 Trainable params: 0 (0.00 B)

 Non-trainable params: 0 (0.00 B)
```

Compile and print out summary

```
simple_rnn_model = Sequential([
    layers.Embedding(input_dim=max_tokens, output_dim=64,
mask_zero=True),
    layers.SimpleRNN(64),
    layers.Dense(1, activation='sigmoid')
])

simple_rnn_model.compile(
    loss = 'binary_crossentropy',
    optimizer = 'adam',
    metrics = ['accuracy']
)

print("Simple RNN Summary")
simple_rnn_model.summary()

lstm_model = Sequential([
    layers.Input(shape=(max_len,)),
    layers.Embedding(input_dim=max_tokens, output_dim=64,
mask_zero=True),
    layers.LSTM(64),
    layers.Dense(1, activation='sigmoid')
])
```

```
lstm_model.compile(
    loss = 'binary_crossentropy',
    optimizer = 'adam',
    metrics = ['accuracy']
)

print("LSTM Summary")
lstm_model.summary()
```

```
Simple RNN Summary

Model: "sequential_2"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃         Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━┩
│ embedding_2 (Embedding)              │ ?                           │   0 (unbuilt) │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ simple_rnn_1 (SimpleRNN)             │ ?                           │   0 (unbuilt) │
├──────────────────────────────────────┼─────────────────────────────┼─────────────────┤
│ dense_2 (Dense)                      │ ?                           │   0 (unbuilt) │
└──────────────────────────────────────┴─────────────────────────────┴─────────────────┘
```

```
 Total params: 0 (0.00 B)

 Trainable params: 0 (0.00 B)

 Non-trainable params: 0 (0.00 B)

LSTM Summary

Model: "sequential_3"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃         Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━┩
│ embedding_3 (Embedding)              │ (None, 128, 64)             │
```

```
1,920,000 │
├─────────────────────────────────┼────────────────────────────┼──────────
──────┤
│ lstm_1 (LSTM)                   │ (None, 64)                 │
33,024 │
├─────────────────────────────────┼────────────────────────────┼──────────
──────┤
│ dense_3 (Dense)                 │ (None, 1)                  │
65 │
└─────────────────────────────────┴────────────────────────────┴──────────
──────┘

 Total params: 1,953,089 (7.45 MB)

 Trainable params: 1,953,089 (7.45 MB)

 Non-trainable params: 0 (0.00 B)
```

Task 1.2: Train your model

```
X_train_vec = vectorizer(X_train)
X_test_vec = vectorizer(X_test)

print("Train Vectorized Shape", X_train_vec.shape)
print("Val Vectorized Shape", X_test_vec.shape)

history_rnn = simple_rnn_model.fit(
    X_train_vec,
    y_train,
    validation_data=(X_test_vec, y_test),
    epochs=5,
    batch_size=128,
)

history_lstm = lstm_model.fit(
    X_train_vec,
    y_train,
    validation_data=(X_test_vec, y_test),
    epochs=5,
    batch_size=128,
)
```

```
Train Vectorized Shape (37500, 128)
Val Vectorized Shape (12500, 128)
Epoch 1/5
293/293 ━━━━━━━━━━━━━━━━━━━━ 10s 22ms/step - accuracy: 0.6985 - loss:
0.5682 - val_accuracy: 0.8254 - val_loss: 0.4104
Epoch 2/5
293/293 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step - accuracy: 0.8815 - loss:
0.3047 - val_accuracy: 0.8108 - val_loss: 0.4179
```

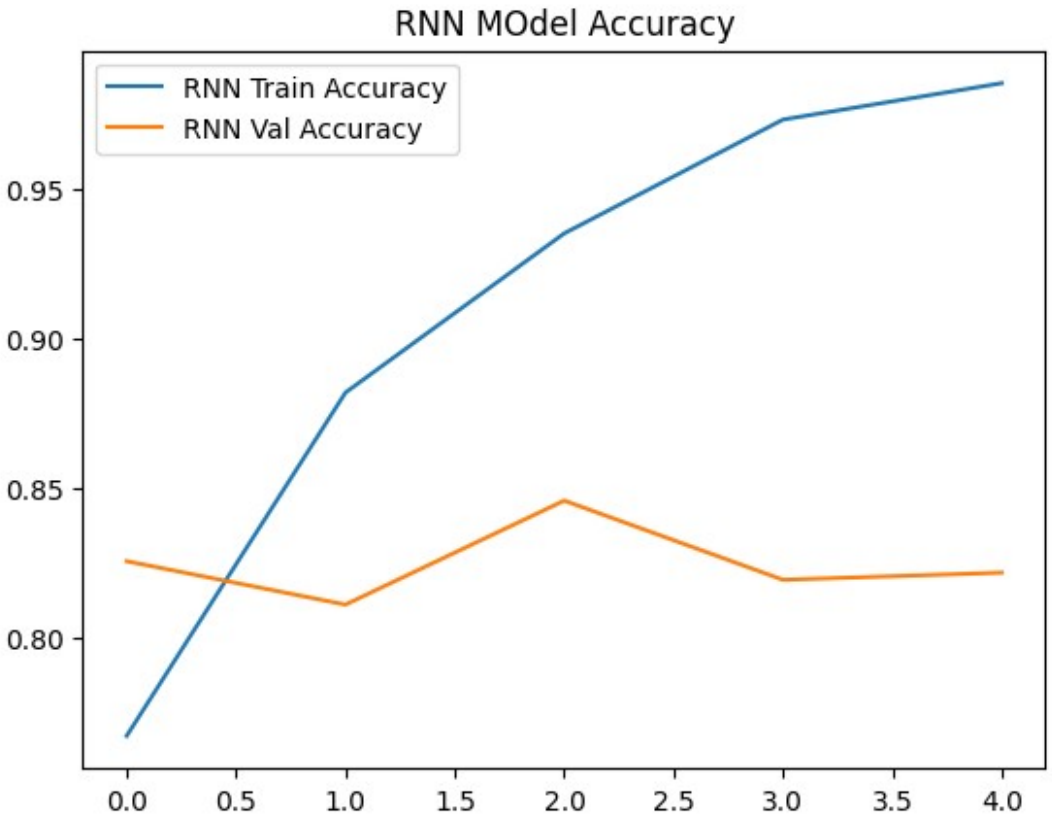
```
Epoch 3/5
293/293 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step - accuracy: 0.9366 - loss:
0.1818 - val_accuracy: 0.8457 - val_loss: 0.4217
Epoch 4/5
293/293 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step - accuracy: 0.9756 - loss:
0.0751 - val_accuracy: 0.8191 - val_loss: 0.4999
Epoch 5/5
293/293 ━━━━━━━━━━━━━━━━━━━━ 4s 13ms/step - accuracy: 0.9860 - loss:
0.0458 - val_accuracy: 0.8215 - val_loss: 0.5715
Epoch 1/5
293/293 ━━━━━━━━━━━━━━━━━━━━ 8s 15ms/step - accuracy: 0.7204 - loss:
0.5178 - val_accuracy: 0.8608 - val_loss: 0.3279
Epoch 2/5
293/293 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.9079 - loss:
0.2436 - val_accuracy: 0.8580 - val_loss: 0.3336
Epoch 3/5
293/293 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.9428 - loss:
0.1630 - val_accuracy: 0.8520 - val_loss: 0.3752
Epoch 4/5
293/293 ━━━━━━━━━━━━━━━━━━━━ 3s 11ms/step - accuracy: 0.9589 - loss:
0.1173 - val_accuracy: 0.8266 - val_loss: 0.4760
Epoch 5/5
293/293 ━━━━━━━━━━━━━━━━━━━━ 4s 14ms/step - accuracy: 0.9724 - loss:
0.0850 - val_accuracy: 0.8436 - val_loss: 0.5968
```

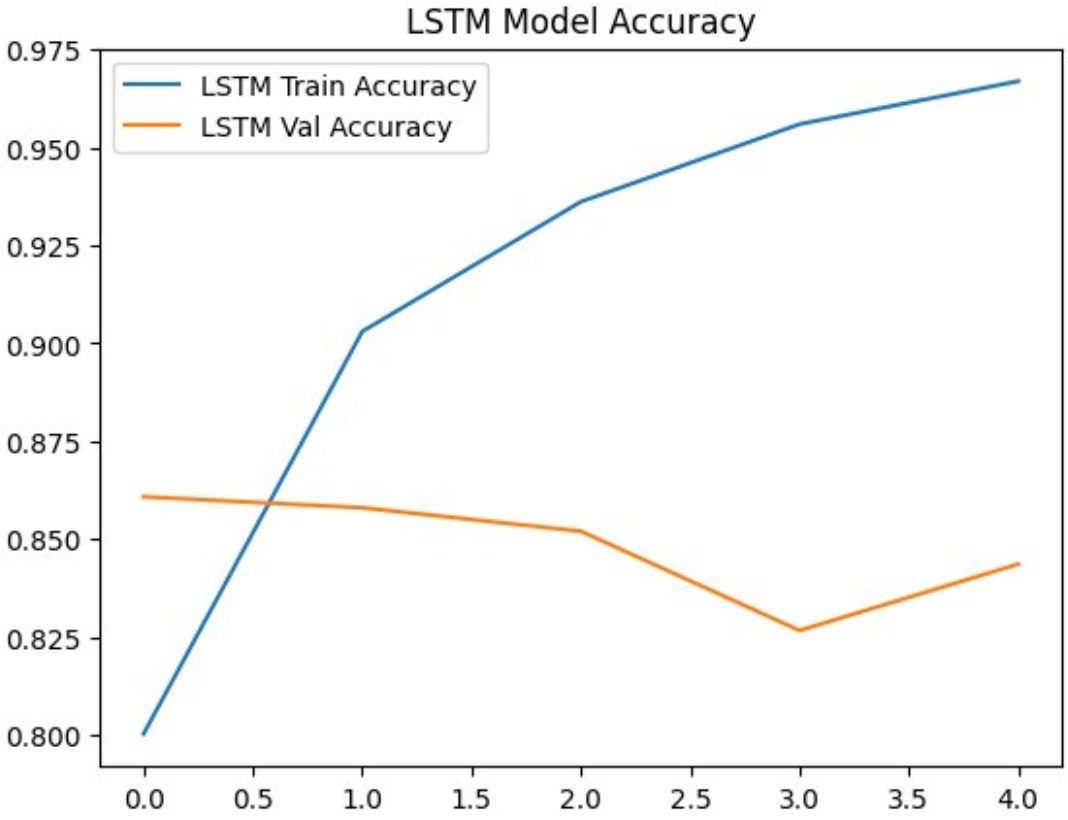
Task 1.3 Evaluate your models

```
#Plotting of the models

plt.plot(history_rnn.history['accuracy'], label='RNN Train Accuracy')
plt.plot(history_rnn.history['val_accuracy'], label='RNN Val
Accuracy')
plt.legend()
plt.title("RNN MOdel Accuracy")
plt.show()


plt.plot(history_lstm.history['accuracy'], label='LSTM Train
Accuracy')
plt.plot(history_lstm.history['val_accuracy'], label='LSTM Val
Accuracy')
plt.legend()
plt.title("LSTM Model Accuracy")
plt.show()
```


RNN MOdel Accuracy
RNN Train Accuracy
RNN Val Accuracy
0.95
0.90
0.85
0.80
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0


LSTM Model Accuracy
LSTM Train Accuracy
LSTM Val Accuracy
0.975
0.950
0.925
0.900
0.875
0.850
0.825
0.800
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

Task 2: BERT

Task 2.0: Install the trasformers package from HuggingFace

Hugging Face is an American company and an open-source community that provides tools, pre-trained machine learning models, and datasets, primarily focused on natural language processing (NLP) and other AI fields like computer vision and audio

Note: You may need to restart the runtime after installing the packages. Then, mount your Google Drive and rerun Task 0.

```
import tensorflow as tf
import keras
# import transformers

print(f"tensorflow: {tf.__version__}")
print(f"keras: {keras.__version__}")
# print(f"transformers: {transformers.__version__}")

#before task 2
#tensorflow: 2.19.1
#keras: 3.11.2

#after task
# tensorflow: 2.20.0
# keras: 3.12.0

tensorflow: 2.19.0
keras: 3.10.0

# !pip install -U "tensorflow==2.19.1" "keras==3.11.2"

!pip install -U "tensorflow>=2.17,<2.21" "keras>=3.3.0" "tf-
keras>=2.17" "transformers>=4.44.0"

import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("IMDB Dataset.csv")

data.head()

from sklearn.preprocessing import LabelEncoder
import re
from sklearn.model_selection import train_test_split

labels = data["sentiment"]
enc = LabelEncoder()
labels_encoded = enc.fit_transform(labels)
data["labels_encoded"] = labels_encoded
data.head()
```

```
def clean_text(text):
    text = re.sub(r"<br\s*/?>", " ", text)  # remove <br> or <br />
    text = re.sub(r" ", " ", text)
    text = re.sub(r"http\S+", " ", text)    # remove URLs
    text = re.sub(r"@\w+", " ", text)       # remove @mentions
    text = re.sub(r"[^A-Za-z0-9\s.,!?']", " ", text)# strip symbols
    text = re.sub(r"\s+", " ", text).strip()
    return text

data["review"] = data["review"].apply(clean_text)

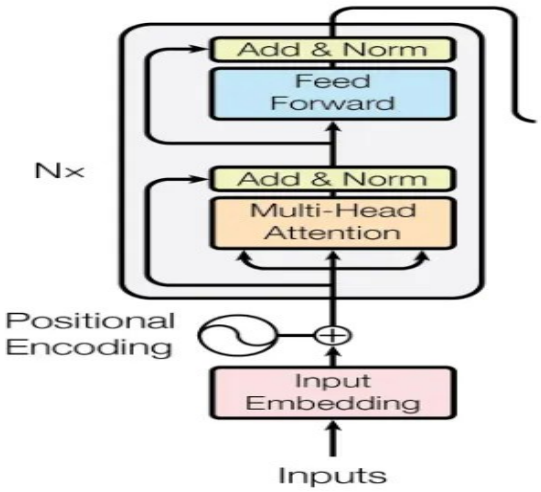
if "labels_encoded" not in data.columns:
     enc = LabelEncoder()
     data["labels_encoded"] = enc.fit_transform(data["sentiment"])

X_train, X_test, y_train, y_test = train_test_split(
     data["review"], data["labels_encoded"], test_size=0.25,
random_state=42)

import tf_keras as keras
import tensorflow as tf
from transformers import TFDistilBertForSequenceClassification,
AutoTokenizer
```

Task 2.1: Load the destilbert model

In this part of the assignment, we will use a smaller and faster version of BERT called DistilBERT. It is significantly more lightweight and trains much faster than the original BERT model. Here is the illustration of its architecture:




```
model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

```
model =
TFDistilBertForSequenceClassification.from_pretrained(model_name,
num_labels=2, from_pt=True)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
```


```
{"model_id":"5221dec4ab18443985cb993cfd84ad5c","version_major":2,"vers
ion_minor":0}
```

```
{"model_id":"8d7bd24703094552a16ab39f1e1dd34b","version_major":2,"vers
ion_minor":0}
```

```
{"model_id":"d7ff9466c91946e88f4cc960ffc687ff","version_major":2,"vers
ion_minor":0}
```

```
{"model_id":"90895c2aeb914dbbb48a35d07188c686","version_major":2,"vers
ion_minor":0}
```

```
{"model_id":"f2dea91c2f6a4f3f8380322cf22c970e","version_major":2,"vers
ion_minor":0}
```


```
TensorFlow and JAX classes are deprecated and will be removed in
Transformers v5. We recommend migrating to PyTorch classes or pinning
your version of Transformers.
Some weights of the PyTorch model were not used when initializing the
TF 2.0 model TFDistilBertForSequenceClassification:
['vocab_projector.weight', 'vocab_projector.bias',
'vocab_layer_norm.bias', 'vocab_transform.weight',
'vocab_layer_norm.weight', 'vocab_transform.bias']
- This IS expected if you are initializing
TFDistilBertForSequenceClassification from a PyTorch model trained on
another task or with another architecture (e.g. initializing a
TFBertForSequenceClassification model from a BertForPreTraining
model).
- This IS NOT expected if you are initializing
TFDistilBertForSequenceClassification from a PyTorch model that you
expect to be exactly identical (e.g. initializing a
TFBertForSequenceClassification model from a
BertForSequenceClassification model).
Some weights or buffers of the TF 2.0 model
TFDistilBertForSequenceClassification were not initialized from the
PyTorch model and are newly initialized: ['pre_classifier.weight',
```

```
'pre_classifier.bias', 'classifier.weight', 'classifier.bias']
You should probably TRAIN this model on a down-stream task to be able
to use it for predictions and inference.
```

Task 2.2: Split the training data to Validation and training

```
from sklearn.model_selection import train_test_split

X_train, X_val, y_train, y_val = train_test_split(X_train, y_train,
test_size=0.2)
```

Task 2.3: Tokenize

You have to tokenize your inputs again, use the AutoTokenizer that we imported earlier.

```
max_len   = 128
batch_sz  = 64
epochs    = 3

enc_train = tokenizer(
    X_train.astype(str).tolist(), truncation=True,
padding="max_length",
    max_length=max_len, return_tensors="np"
)
enc_val = tokenizer(
    X_val.astype(str).tolist(), truncation=True, padding="max_length",
    max_length=max_len, return_tensors="np"
)
```

Create datasets

This step wraps your input features (enc_train, e.g., input_ids and attention_mask) and labels (y_train) into a TensorFlow-native streaming dataset.

The TensorFlow Dataset API provides an efficient way to feed data to your model:

- Streams data efficiently from RAM or disk.
- Performs batching, shuffling, and preprocessing in parallel on the CPU.
- Keeps the GPU busy by prefetching the next batch while training.
- Integrates seamlessly with Hugging Face's encoded token dictionaries.

```
ds_train = tf.data.Dataset.from_tensor_slices((dict(enc_train),
y_train))
ds_train = ds_train.map(lambda x, y: (x, tf.cast(tf.squeeze(y),
tf.int32))) \
                   .shuffle(10000).batch(batch_sz).prefetch(tf.data.AU
```

```
TOTUNE)

ds_val = tf.data.Dataset.from_tensor_slices((dict(enc_val), y_val))
ds_val = ds_val.map(lambda x, y: (x, tf.cast(tf.squeeze(y),
tf.int32))) \
               .batch(batch_sz).prefetch(tf.data.AUTOTUNE)
```

Task 2.4: Create and Compile model

```
optimizer = tf.keras.optimizers.Adam(learning_rate=5e-7) #massive
overfit on higher lr
loss = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
metrics = [tf.keras.metrics.SparseCategoricalAccuracy()]


model.compile(
    optimizer=optimizer,
    loss=loss,
    metrics=metrics
)
```

Task 2.5: Train your model

```
epochs = 3
batch_size = 32

history_distilbert = model.fit(ds_train, validation_data=ds_val,
epochs=epochs, batch_size=batch_size)

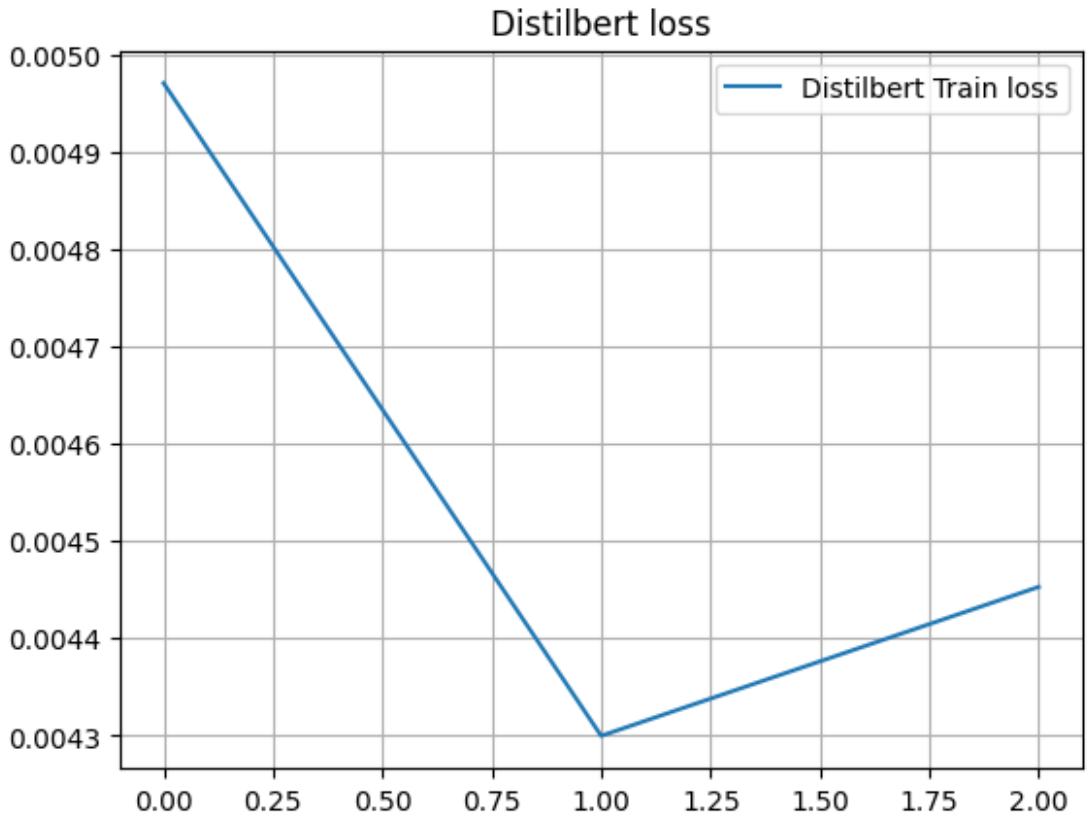
Epoch 1/3
469/469 [==============================] - 423s 857ms/step - loss:
0.0050 - sparse_categorical_accuracy: 0.9988 - val_loss: 0.6716 -
val_sparse_categorical_accuracy: 0.8821
Epoch 2/3
469/469 [==============================] - 402s 858ms/step - loss:
0.0043 - sparse_categorical_accuracy: 0.9991 - val_loss: 0.6709 -
val_sparse_categorical_accuracy: 0.8827
Epoch 3/3
469/469 [==============================] - 401s 855ms/step - loss:
0.0045 - sparse_categorical_accuracy: 0.9991 - val_loss: 0.6739 -
val_sparse_categorical_accuracy: 0.8828
```

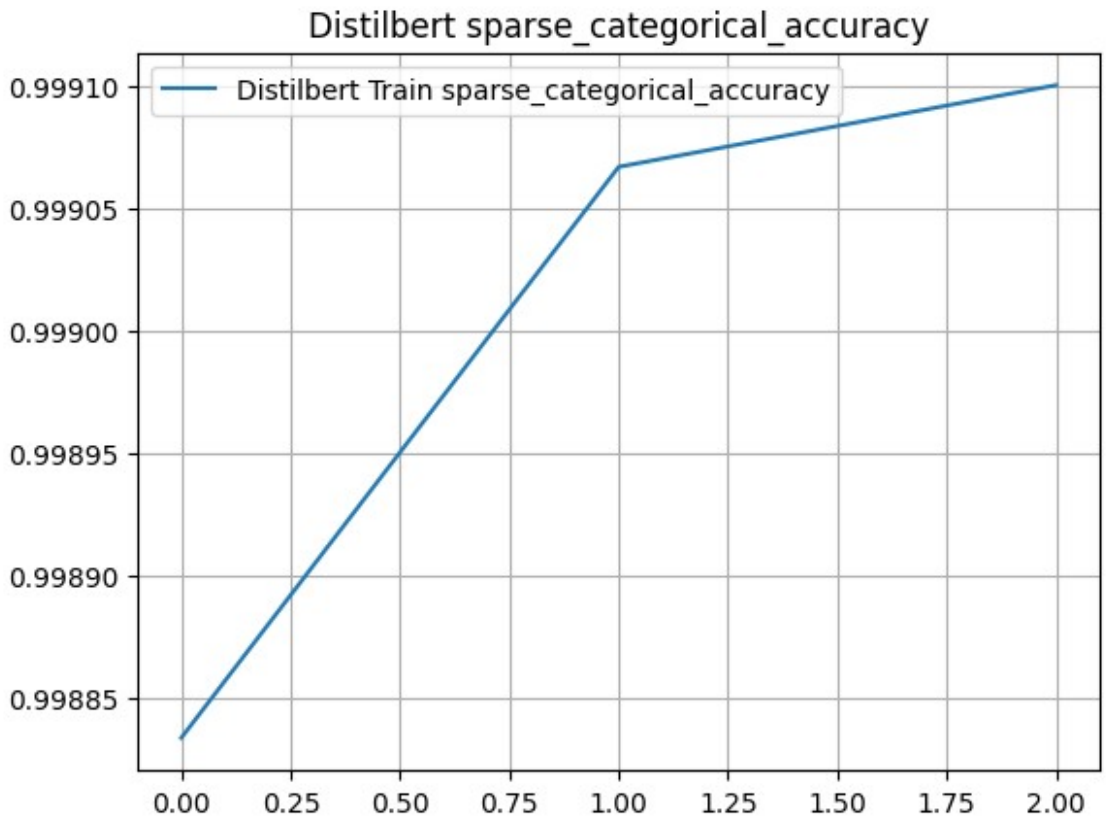
Task 2.6: Evaluate

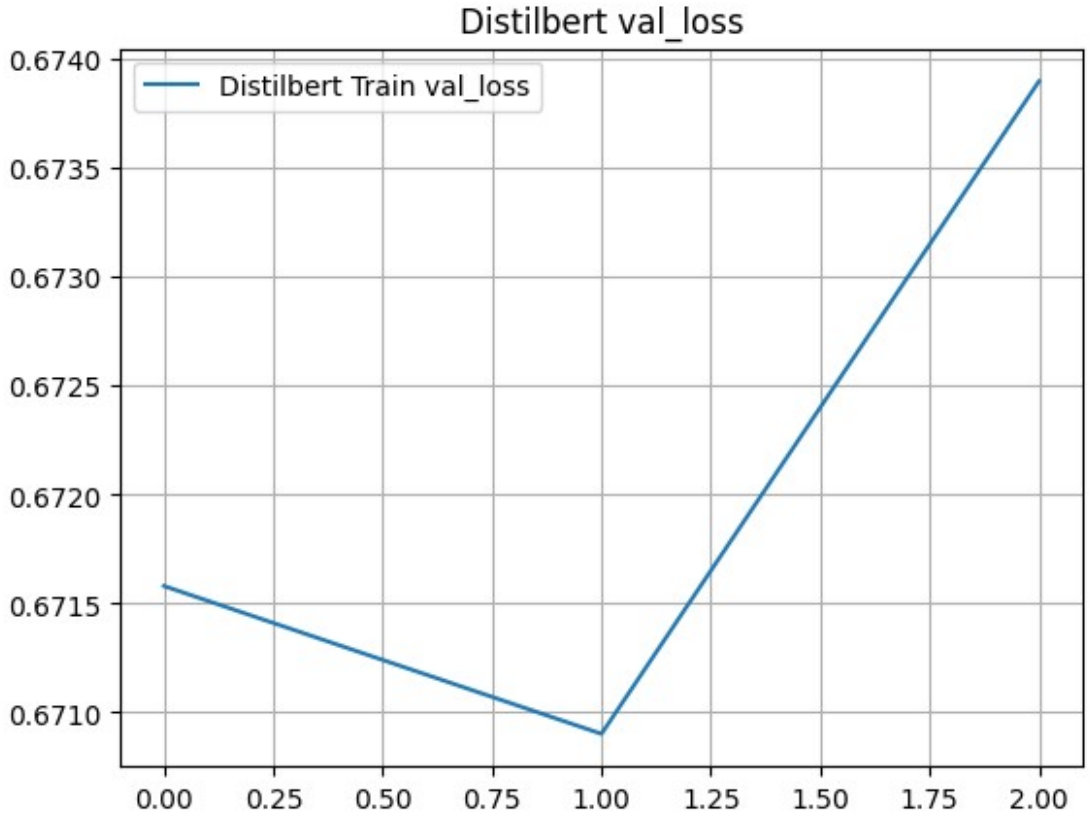
```
dicc = history_distilbert.history

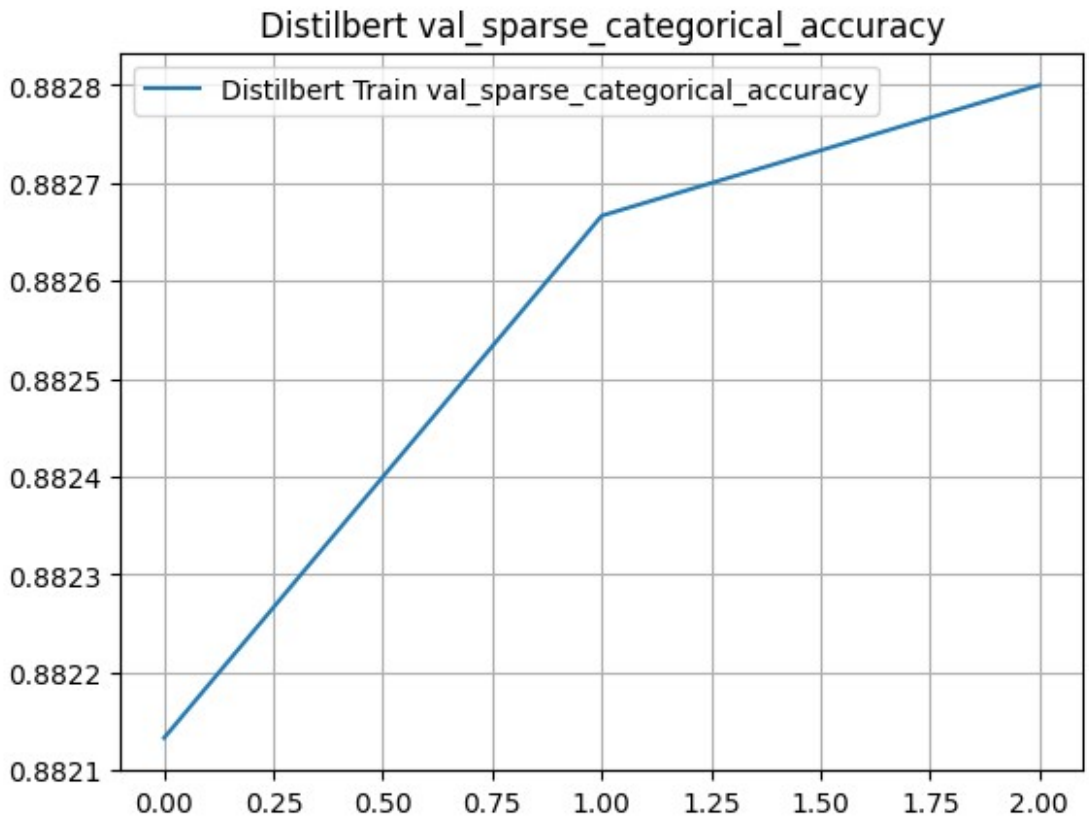
for key in dicc:
  plt.plot(dicc[key], label=f'Distilbert Train {key}')
  plt.legend()
```

```
plt.grid()
plt.title(f"Distilbert {key}")
plt.show()
```




Distilbert sparse_categorical_accuracy
Distilbert Train sparse_categorical_accuracy
0.99910
0.99905
0.99900
0.99895
0.99890
0.99885
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00


Distilbert val_loss
Distilbert Train val_loss
0.6740
0.6735
0.6730
0.6725
0.6720
0.6715
0.6710
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00



Task 3: Compare

3.1: Compare the results of the two models. What could explain the differences in their performance?

Answer: our RNN has to start from random weights while destilbert is pretrained by competent data scientists.

3.2: Why did we need to tokenize the inputs again using a different tokenizer in Task 2.3?

Answer: RNNs dont depend on how the text is tokenized since they go from token to token, but the pretrained weights of the destilbert require the tokens to use the same vocabulary, or else the weights wont work properly.

3.3: Briefly explain the main difference between RNNs and Transformer architectures.

Answer: The main difference is RNNs process tokens sequencially, while destilbert process them parallellically. The memory of RNN is hidden state while destilbert is attention based.

Task 4: Reading Exercise (Optional)

If you truly want to understand the logic behind ChatGPT and the motivation for Transformers, I encourage you to read the 2017 paper "Attention Is All You Need." In this paper, engineers from Google explain in detail how the architecture was built and why it represents such a major breakthrough. wow